

编译原理实验报告

- 实验题目：Clang 静态检测项开发与自动测试 (MISRA C:2012)
- 开发环境：Windows + CLion + MSYS2 (UCRT64)
- LLVM/Clang 版本：llvmorg-17.0.5
- 规则组：MISRA C:2012 Rule 7.1 / 8.11 / 15.4

本实验分为三个阶段：

- 阶段一：基于 LLVM/Clang Static Analyzer 开发 3 条 MISRA 规则的检测项，并完成注册与编译。
- 阶段二：安装并配置 nanobot，为后续自动化流程提供大模型能力。
- 阶段三：通过自动化脚本生成测试用例、执行静态检测、并在工作副本中完成缺陷修复。

阶段一：CLion + MSYS2(UCRT64) 构建与检测项开发

1.基础环境准备 (MSYS2 + CLion)

1.安装编译所需工具链

```
MSYS2 UCRT64 Shell
86180@zhangf:~$ echo "=== 检查工具链 ===" && \
gcc --version && \
g++ --version && \
cmake --version && \
ninja --version && \
echo "=== 所有工具检查完成 ==="
=== 检查工具链 ===
gcc.exe (Rev2, Built by MSYS2 project) 16.1.0
Copyright (C) 2026 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

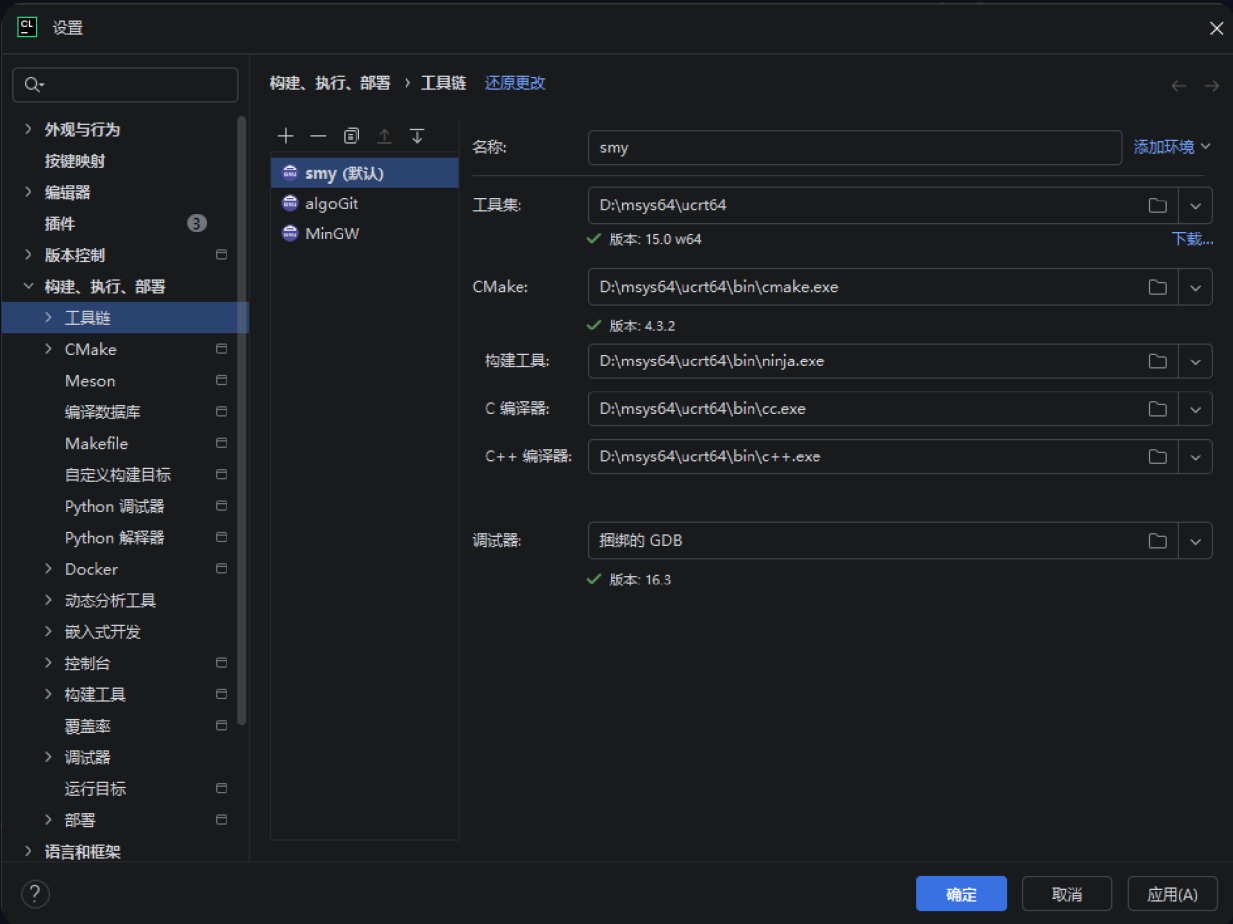
g++.exe (Rev2, Built by MSYS2 project) 16.1.0
Copyright (C) 2026 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

cmake version 4.3.2

CMake suite maintained and supported by Kitware (kitware.com/cmake).
1.13.2
=== 所有工具检查完成 ===
86180@zhangf:~$ |
```

在 MSYS2 UCRT64 环境中安装用于编译 LLVM/Clang 的基础工具链（编译器、CMake、Ninja 等）。实验中采用 CLion 的 Toolchains 调用 MSYS2 的编译工具链，从而在 Windows 上实现命令行方式的 CMake + Ninja 构建流程。

2.配置 CLion Toolchains:



在 CLion 中将 Toolchain 指向 MSYS2 的 UCRT64 环境（如 `D:\msys64\ucrt64`），并确保 CMake、Ninja、C/C++ 编译器均能被正确识别。后续 CLion 的 CMake Profile 使用该 Toolchain 即可完成 LLVM/Clang 的配置与构建。

2. 克隆 LLVM 源码

📁 .ci	2023/11/14 16:22	文件夹	
📁 .github	2023/11/14 16:22	文件夹	
📁 .idea	2026/5/9 21:37	文件夹	
📁 bolt	2023/11/14 16:22	文件夹	
📁 clang	2023/11/14 16:22	文件夹	
📁 clang-tools-extra	2023/11/14 16:22	文件夹	
📁 cmake	2023/11/14 16:22	文件夹	
📁 compiler-rt	2023/11/14 16:22	文件夹	
📁 cross-project-tests	2023/11/14 16:22	文件夹	
📁 flang	2023/11/14 16:22	文件夹	
📁 libc	2023/11/14 16:22	文件夹	
📁 libclc	2023/11/14 16:22	文件夹	
📁 libcoxx	2023/11/14 16:22	文件夹	
📁 libcoxxabi	2023/11/14 16:22	文件夹	
📁 libunwind	2023/11/14 16:22	文件夹	
📁 lld	2023/11/14 16:22	文件夹	
📁 lldb	2023/11/14 16:22	文件夹	
📁 llvm	2023/11/14 16:22	文件夹	
📁 llvm-libgcc	2023/11/14 16:22	文件夹	
📁 mlir	2023/11/14 16:22	文件夹	
📁 openmp	2023/11/14 16:22	文件夹	
📁 polly	2023/11/14 16:22	文件夹	
📁 pstl	2023/11/14 16:22	文件夹	
📁 runtimes	2023/11/14 16:22	文件夹	
📁 third-party	2023/11/14 16:22	文件夹	
📁 utils	2023/11/14 16:22	文件夹	
📄 .arcconfig	2023/11/14 16:22	ARCCONFIG 文件	1 KB
📄 .arclint	2023/11/14 16:22	ARCLINT 文件	1 KB
📄 .clang-format	2023/11/14 16:22	CLANG-FORMAT 文件	1 KB
📄 .clang-tidy	2023/11/14 16:22	CLANG-TIDY 文件	2 KB
📄 .git-blame-ignore-revs	2023/11/14 16:22	GIT-BLAME-IGNORE-REVS 文...	3 KB
📄 .gitignore	2023/11/14 16:22	txtfile	3 KB
📄 .mailmap	2023/11/14 16:22	MAILMAP 文件	2 KB
📄 CONTRIBUTING.md	2023/11/14 16:22	Markdown File	1 KB
📄 LICENSE.TXT	2023/11/14 16:22	文本文档	15 KB
📄 README.md	2023/11/14 16:22	Markdown File	2 KB
📄 SECURITY.md	2023/11/14 16:22	Markdown File	1 KB

通过 Git 克隆 `llvmorg-17.0.5` 对应的 LLVM monorepo 源码。为避免构建工具链在 Windows 下因路径编码、空格等产生问题，项目放置在全英文路径中。

3.在 CLion 中导入和配置项目

```

CMake
Release
-- Performing Test HAVE_CXX_FLAG_FSTRICT_ALIASING - Failed
-- Performing Test HAVE_CXX_FLAG_FSTRICT_ALIASING - Success
-- Performing Test HAVE_CXX_FLAG_WNO_DEPRECATED_DECLARATIONS
-- Performing Test HAVE_CXX_FLAG_WNO_DEPRECATED_DECLARATIONS - Success
-- Performing Test HAVE_CXX_FLAG_FNO_EXCEPTIONS
-- Performing Test HAVE_CXX_FLAG_FNO_EXCEPTIONS - Success
-- Performing Test HAVE_CXX_FLAG_WSTRICT_ALIASING
-- Performing Test HAVE_CXX_FLAG_WSTRICT_ALIASING - Success
-- Performing Test HAVE_CXX_FLAG_WD654
-- Performing Test HAVE_CXX_FLAG_WD654 - Failed
-- Performing Test HAVE_CXX_FLAG_WTHREAD_SAFETY
-- Performing Test HAVE_CXX_FLAG_WTHREAD_SAFETY - Failed
-- Performing Test HAVE_CXX_FLAG_COVERAGE
-- Performing Test HAVE_CXX_FLAG_COVERAGE - Success
-- Performing Test HAVE_GNU_POSIX_REGEX
-- Performing Test HAVE_GNU_POSIX_REGEX -- failed to compile
-- Performing Test HAVE_POSIX_REGEX
-- Performing Test HAVE_POSIX_REGEX
-- Performing Test HAVE_POSIX_REGEX -- failed to compile
CMake Warning at D:/MyCodeResource/bianyiyuanli/exec/clang/llvm-project-llvmorg-17.0.5/third-party/benchmark/CMakeLists.txt:308 (message): @使用 AI 编程
Using std::regex with exceptions disabled is not fully supported

-- Performing Test HAVE_STEADY_CLOCK
-- Performing Test HAVE_STEADY_CLOCK
-- Performing Test HAVE_STEADY_CLOCK -- success
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD - Success
-- Found Threads: TRUE
-- Configuring done (76.0s)
CMake Warning in examples/Orcv2CBindingsMCJITLikeMemoryManager/CMakeLists.txt: @使用 AI 编程
The object file directory

D:/MyCodeResource/bianyiyuanli/exec/clang/llvm-project-llvmorg-17.0.5/llvm/cmake-build-release/examples/Orcv2CBindingsMCJITLikeMemoryManager/CMakeFiles/Orcv2CBindingsMCJITLikeMemoryManager.dir/.//

has 209 characters. The maximum full path to an object file is 258
characters (see CMAKE_OBJECT_PATH_MAX). Object file

Orcv2CBindingsMCJITLikeMemoryManager.c.obj

cannot be safely placed under this directory. The build may not work
correctly.

-- Generating done (7.2s)
-- Build files have been written to: D:/MyCodeResource/bianyiyuanli/exec/clang/llvm-project-llvmorg-17.0.5/llvm/cmake-build-release

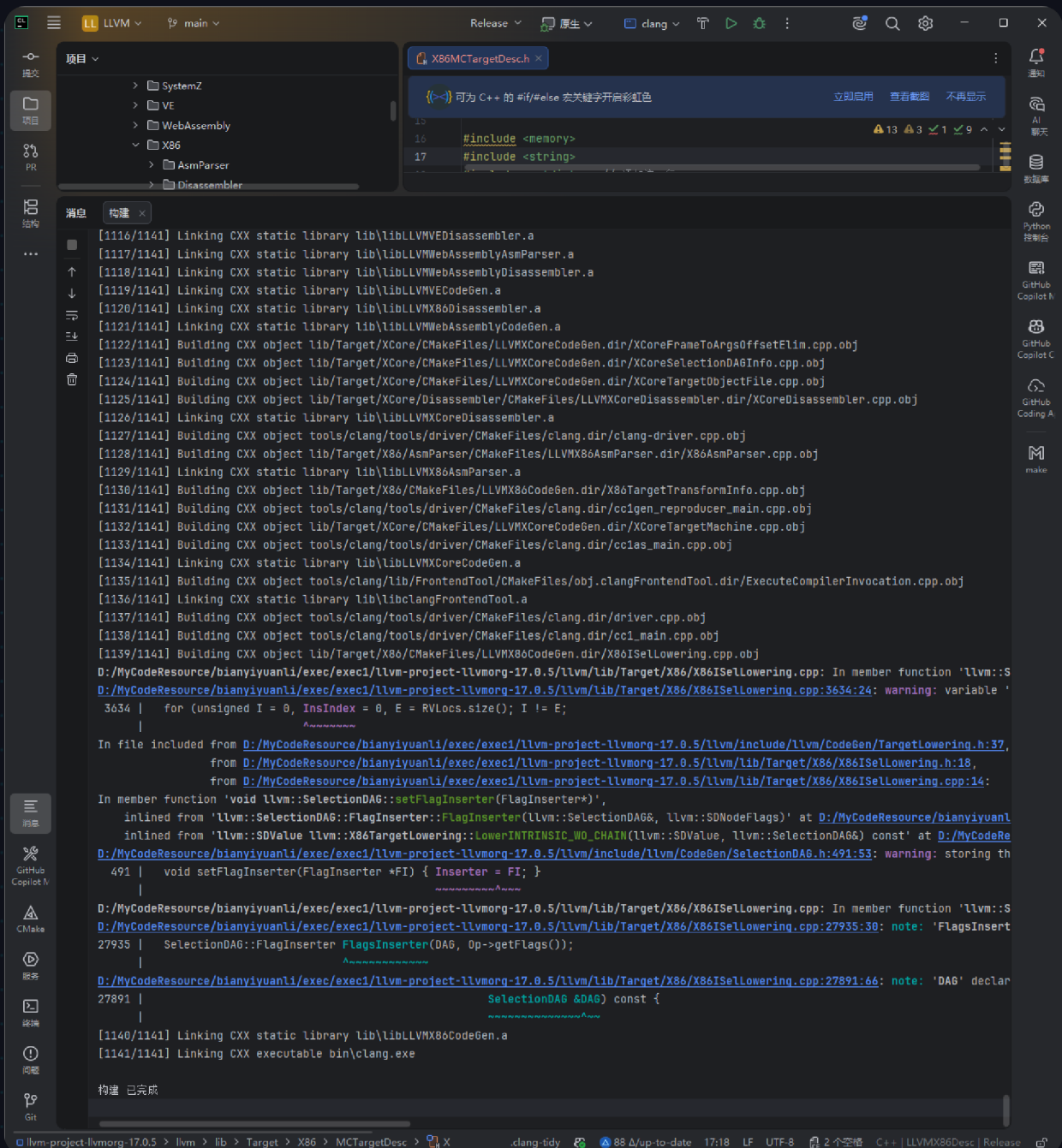
[已完成]
```

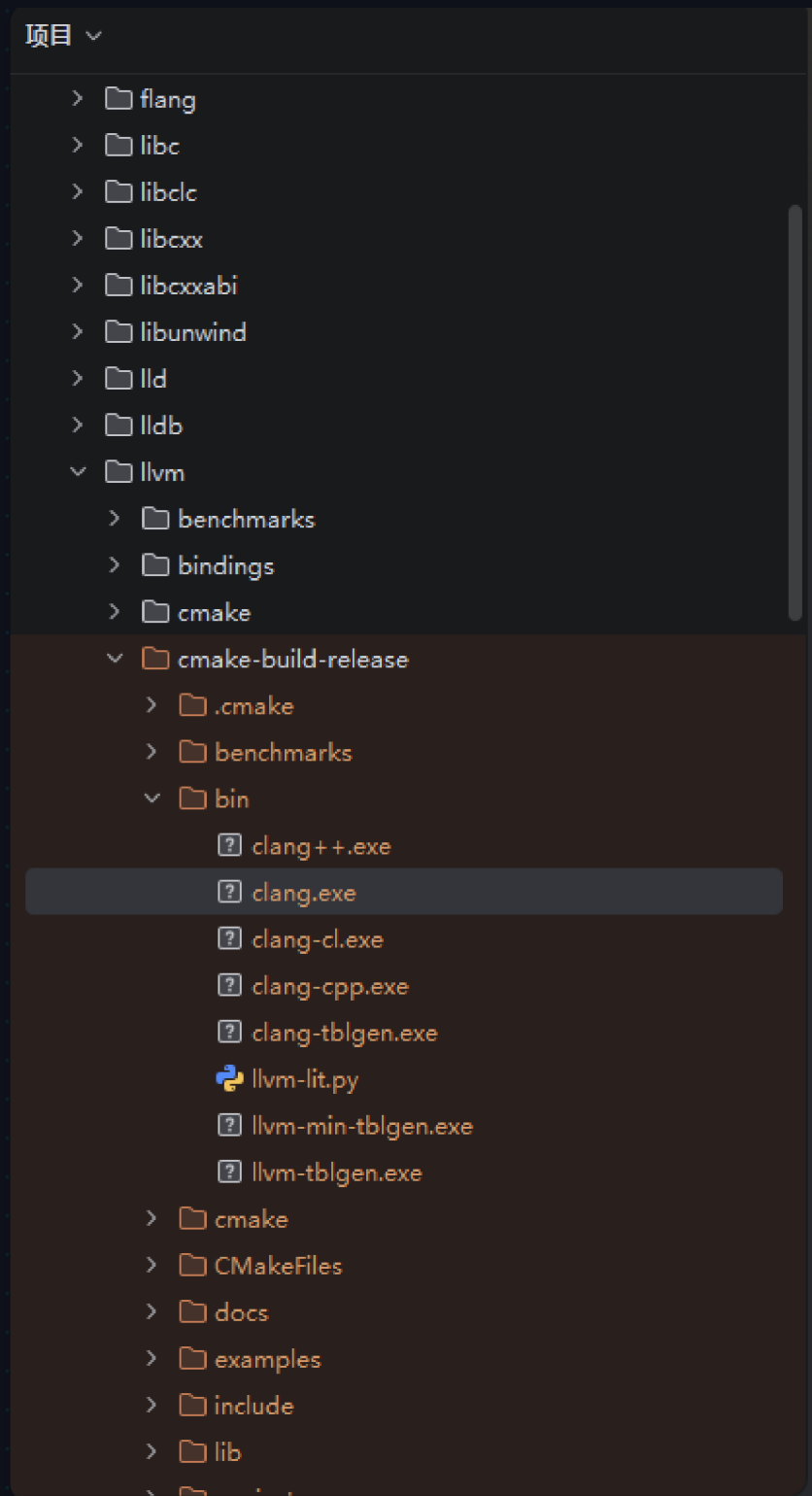
在 CLion 中打开 `llvm-project/llvm` 目录的 `CMakeLists.txt`（注意不是仓库根目录），并在 CMake Options 中启用 Clang 子项目：

```
1 | -DLLVM_ENABLE_PROJECTS="clang"
```

同时选择 Release 构建类型，减少编译体积与耗时。

4. 编译 Clang





在 CLion 中选择 **clang** 目标并进行 Build，成功后生成 **clang.exe**。本实验用于后续自动化检测的 clang 路径示例：

```
1 | D:\MyCodeResource\bianyiyuanli\exec\exec1\llvm-  
project-llvmorg-17.0.5\llvm\cmake-build-  
release\bin\clang.exe
```

为便于提交，已将本阶段构建产物归档到：

- `实验报告/阶段一产物/构建产物/clang.exe`

5. 检测项开发流程 (MISRA C:2012 Rule 7.1, 8.11, 15.4)

I 第一步：创建检测项代码

已经构造：

- `MISRAC_7_1_Checker.cpp` (八进制禁止使用)
- `MISRAC_8_11_Checker.cpp` (具有外部链接的数组声明必须指定大小)
- `MISRAC_15_4_Checker.cpp` (循环内的 `break/goto` 不能超过一个)

提交归档路径：

- `实验报告/阶段一产物/源文件/MISRAC_7_1_Checker.cpp`
- `实验报告/阶段一产物/源文件/MISRAC_8_11_Checker.cpp`
- `实验报告/阶段一产物/源文件/MISRAC_15_4_Checker.cpp`

• R_7_1 (Octal constants shall not be used)

- 检测目标：识别形如 `010`、`077` 的整数字面量（以 `0` 开头且不属于 `0x/0X/0b/0B`），报告 `MISRA.R_7_1` 缺陷。
- 关键实现：在 AST 遍历中访问 `IntegerLiteral`，使用 `Lexer::getSourceText(...)` 提取源码 Token 文本并判断前缀；为避免宏展开引入复杂定位，本实现跳过宏内字面量（`Loc.isMacroID()`）。

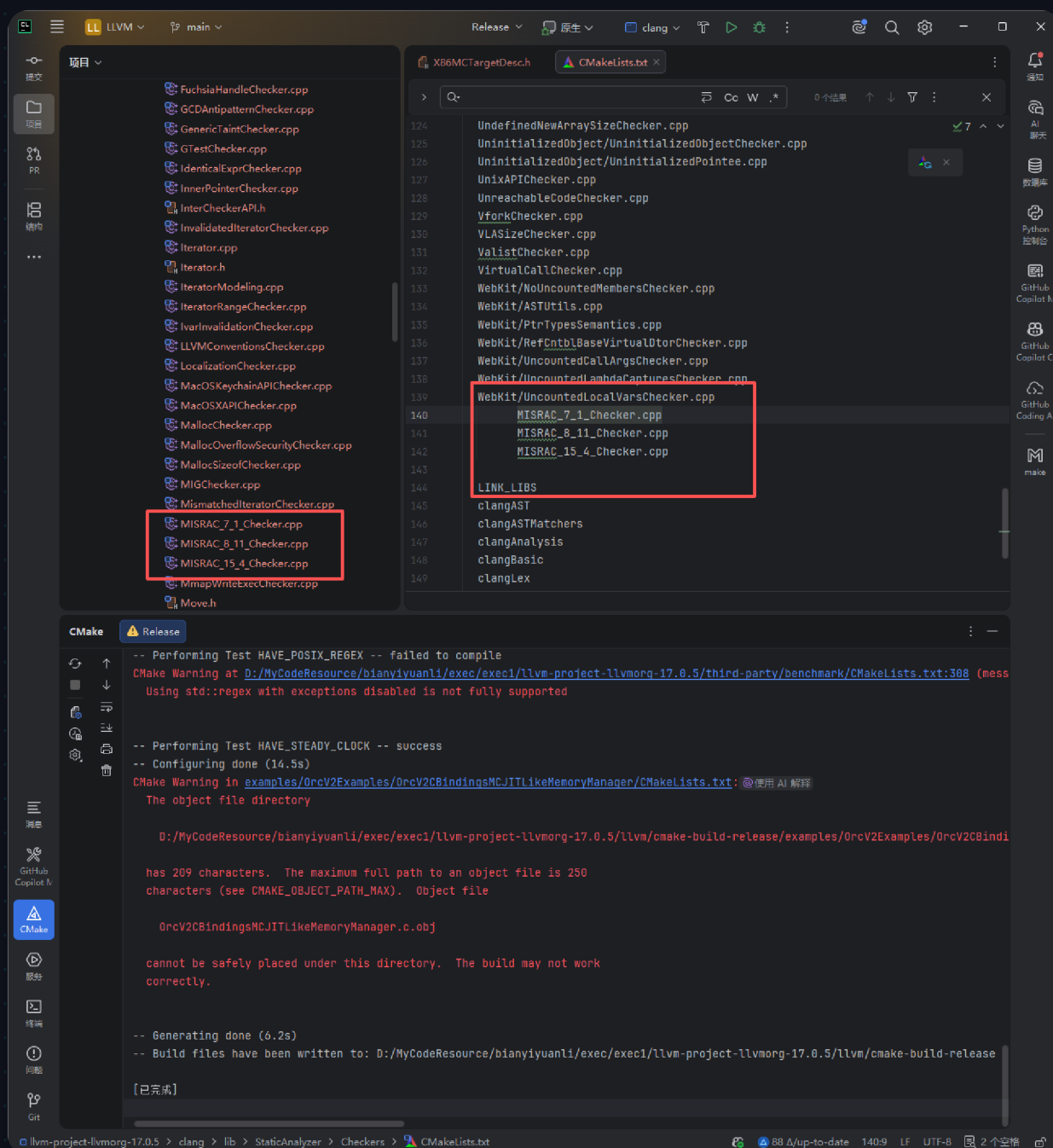
• R_8_11 (外部链接数组声明必须显式指定大小)

- 检测目标：当变量具有外部链接（`ExternalLinkage`）且类型为不完整数组（`IncompleteArrayType`）时，报告 `MISRA.R_8_11` 缺陷。
- 关键实现：基于 `check::ASTDecl<VarDecl>` 检查 `VarDecl`，结合 `VD->getFormalLinkage()` 与 `QualType::isIncompleteArrayType()` 判定。

- R_15_4 (循环内用于终止迭代的 break/goto 不超过 1 个)

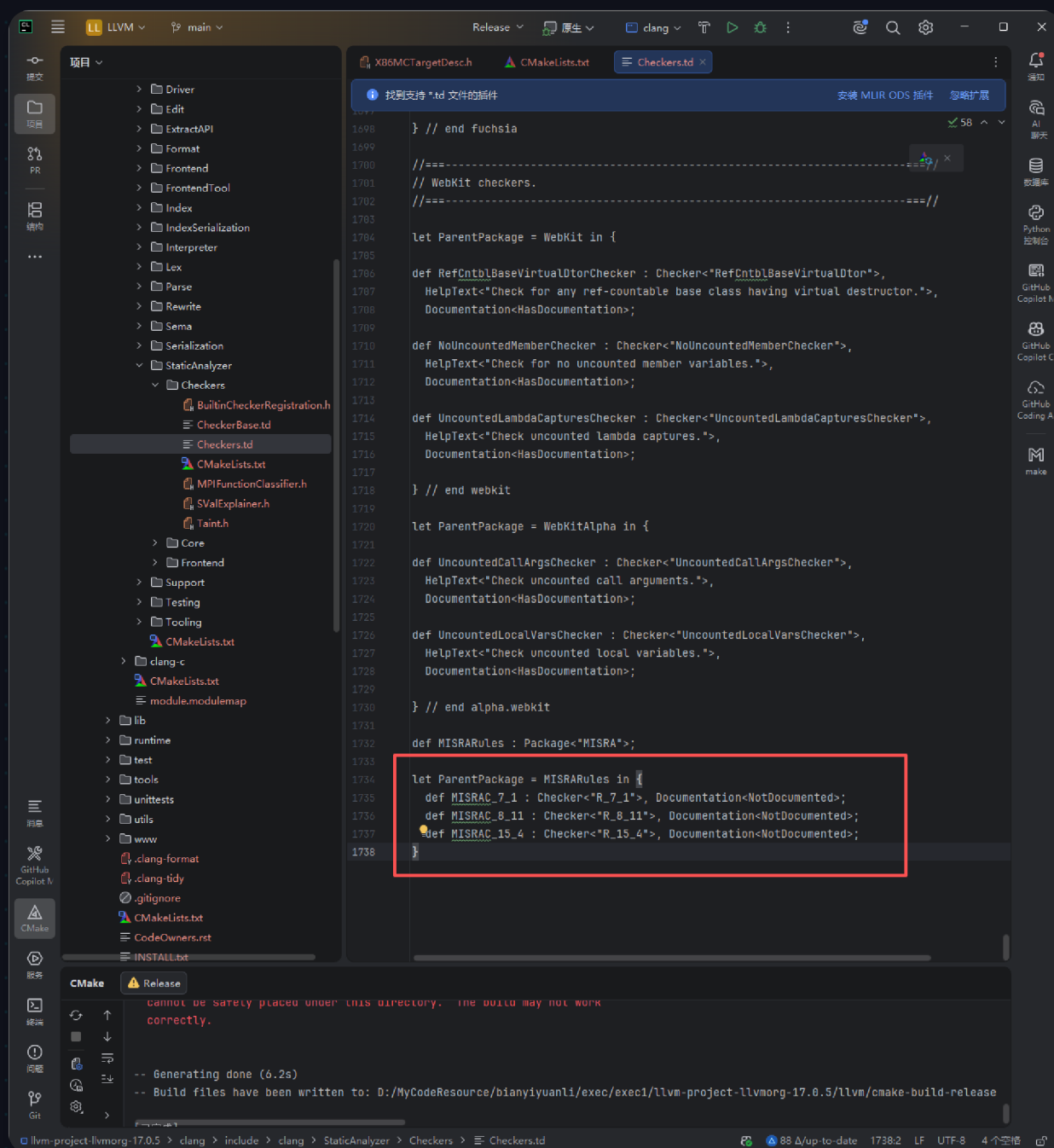
- 检测目标：对 `for/while/do-while` 的循环体统计 `break/goto` 数量，若超过 1 个则报告 `MISRA.R_15_4` 缺陷。
- 关键实现：对每个循环节点提取 body 并递归遍历统计 `BreakStmt` 与 `GotoStmt`；为避免将嵌套循环/嵌套 `switch` 的 `break/goto` 计入外层循环，本实现对嵌套结构截断遍历。

第二步：将文件加入 CMake 构建树



在 `clang/lib/StaticAnalyzer/Checkers/CMakeLists.txt` 中，将 3 个 Checker 源文件加入 `clangStaticAnalyzerCheckers` 的源文件列表，使其参与构建。

第三步：在 Checkers.td 中注册规则



在 `clang/include/clang/StaticAnalyzer/Checkers/Checkers.td` 中注册规则包与检测项：

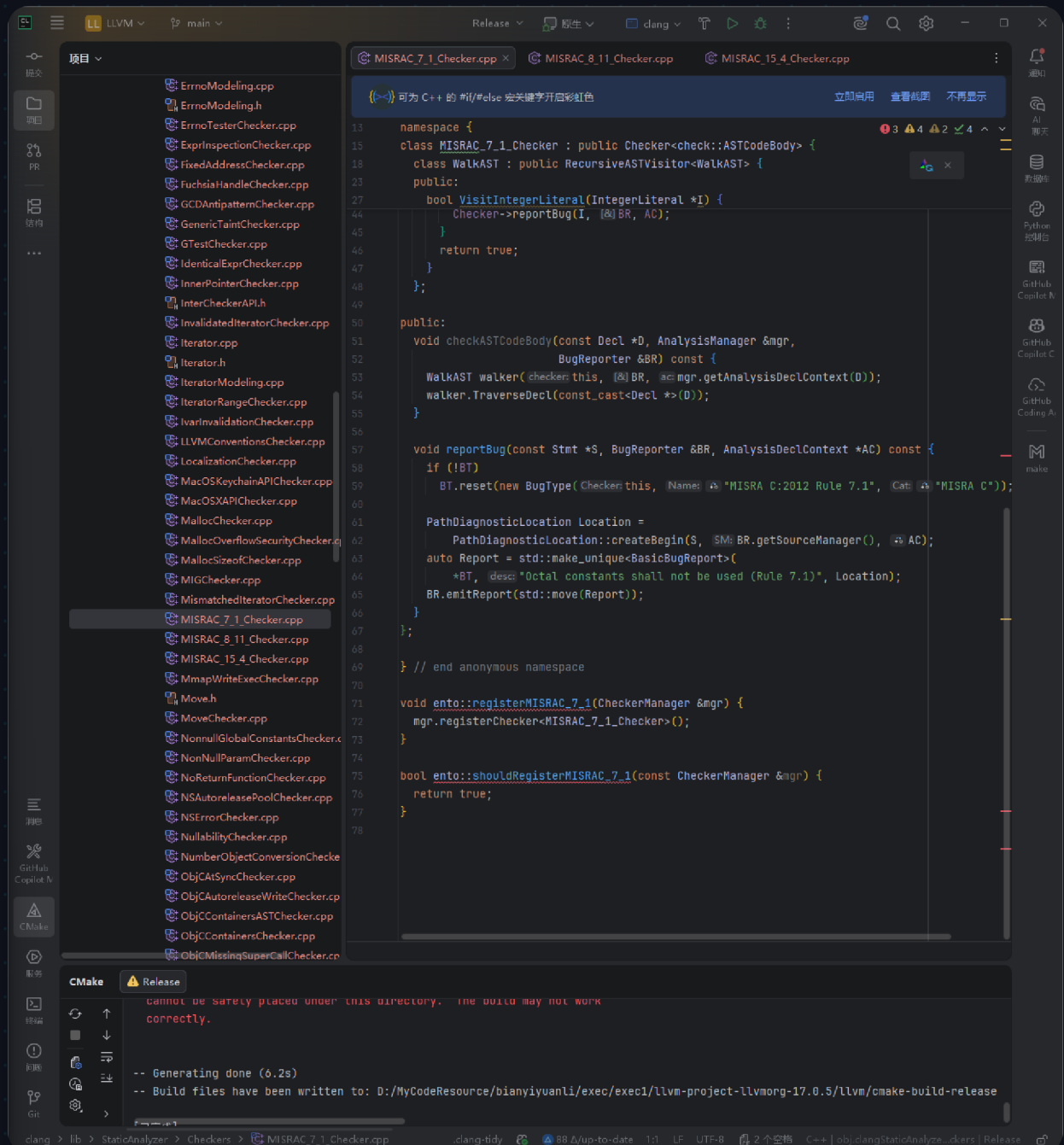
- 规则包： **MISRA**
- 规则 ID： **R_7_1**、**R_8_11**、**R_15_4**

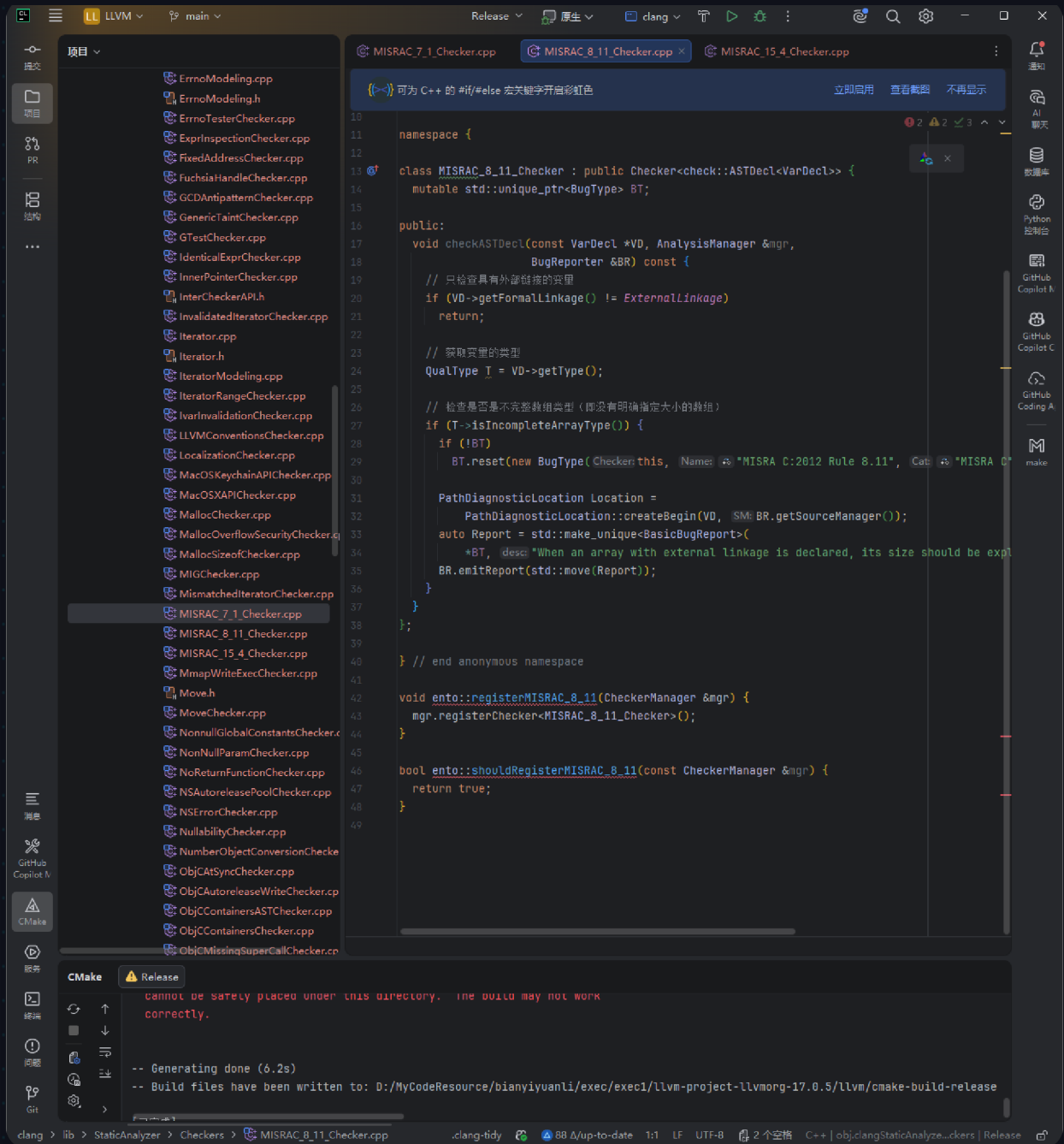
注册完成后，可通过如下方式启用自定义检测项：

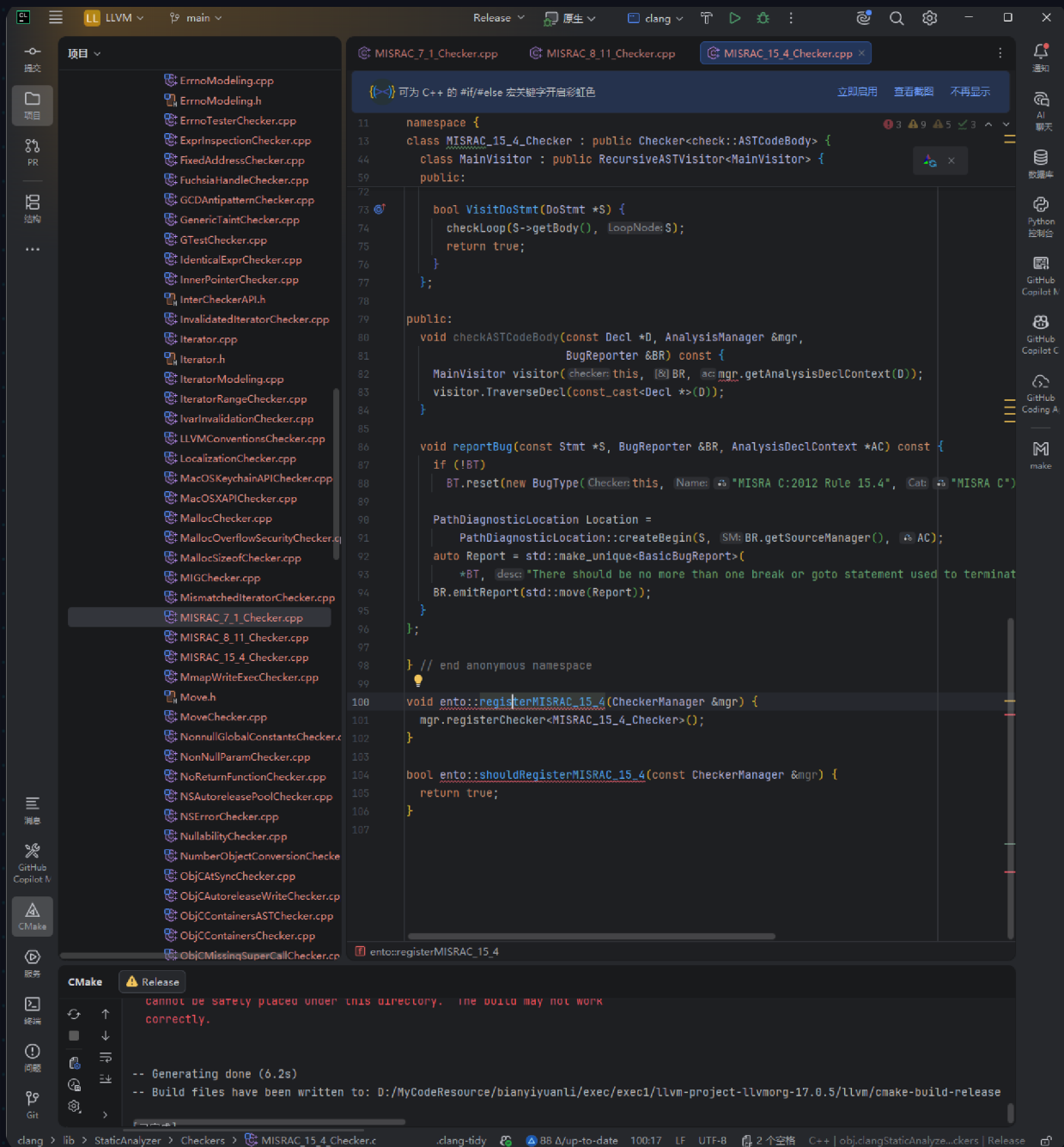
```
bash

1 <clang.exe路径> --analyze -Xanalyzer -analyzer-checker=MISRA.R_7_1 -o NUL test.c
2 <clang.exe路径> --analyze -Xanalyzer -analyzer-checker=MISRA.R_8_11 -o NUL test.c
3 <clang.exe路径> --analyze -Xanalyzer -analyzer-checker=MISRA.R_15_4 -o NUL test.c
```

第四步：在 C++ 代码中双向绑定

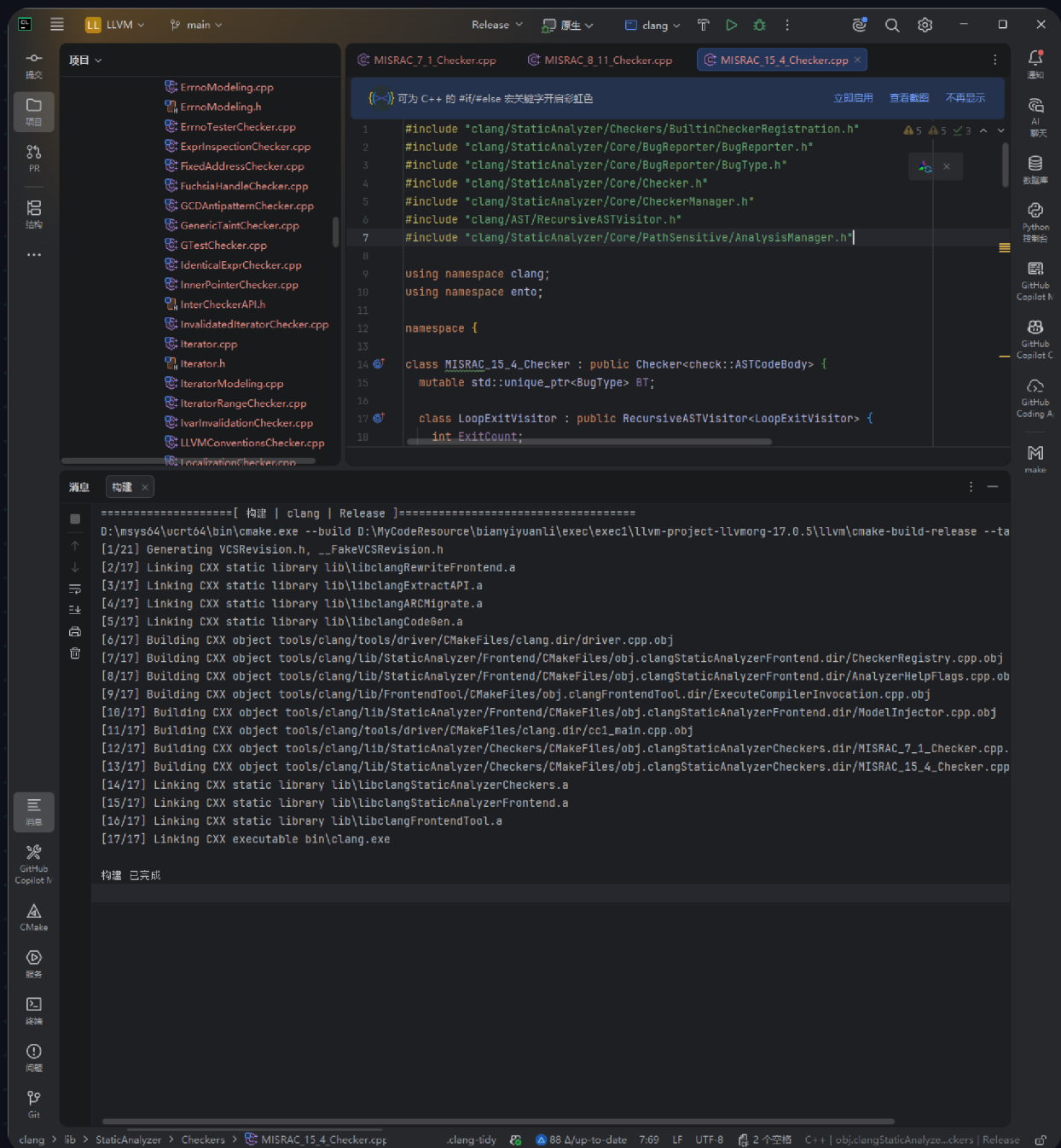






在各 Checker 文件末尾实现 `registerMISRAC_x_x` 与 `shouldRegisterMISRAC_x_x`，并通过 `CheckerManager` 完成注册，从而实现 `Checkers.td` 中规则项与 `.cpp` 实现文件的对应关系。

6. 重新编译与测试



完成新增源码、更新 CMakeLists 与 Checkers.td 后，重新构建 `clang` 目标，使新 Checker 编译进 `clang.exe`。

阶段二：安装 nanobot

1. 安装 nanobot

• 1.1 环境要求

- Python 3.9 或更高版本

• 1.2 安装步骤

打开命令提示符 (cmd) 或 PowerShell, 进入 nanobot-main 目录:

```
bash
1 | cd
   d:\MyCodeResource\bianyiyuanli\exec\exec1\nanobot-
   main
```

安装 nanobot:

```
bash
1 | pip install -e .
```

1: C:\Program Files\PowerShell\7\pwsh.exe X +

Stored in directory: C:\Users\86180\AppData\Local\Temp\pip-ephem-wheel-cache-ge86pgw\wheels\6a\66\4f\3736f8e086f2fc634cdac05678616368058d3690640fee6ceb

Successfully built nanobot-ai

Installing collected packages: pywin32, XlsxWriter, win32-setctime, websockets, websocket-client, wcwidth, urllib3, tzdata, typing-extensions, socksio, sniffio, slack-sdk, six, shellingham, rpds-py, regex, pyyaml, python-multipart, python-dotenv, pypdf, pyjwt, pygments, pycryptodome, pycparser, propcache, primp, platformdirs, Pillow, multidict, msgpack, mdurl, MarkupSafe, lxml, json-repair, jmespath, jiter, idna, httpx-sse, h11, frozenlist, filelock, et-xmlfile, docstring-parser, distro, cssselect, colorama, charset_normalizer, chardet, certifi, bidict, attrs, annotated-types, annotated-doc, aiohappyeyeballs, yarl, wsproto, tzlocal, typing-inspection, tqdm, requests, referencing, python-pptx, python-docx, python-dateutil, pydantic-core, prompt-toolkit, openpyxl, markdown-it-py, lxml_html_clean, loguru, jinja2, httpcore, dulwich, click, cffi, anyio, aiosignal, uvicorn, tiktoken, starlette, slackify-markdown, simple-websocket, rich, requests_toolbelt, questionnaire, pydantic, jsonschema-specifications, httpx, ddgs, cryptography, croniter, botocore, APScheduler, aiohttp, typer, sse-starlette, s3transfer, readability-lxml, qq-botpy, python-telegram-bot, python-engineio, pydantic-settings, openai, oauth-cli-kit, lark-oapi, jsonschema, dingtalk-stream, anthropic, python-socketio, mcp, boto3, nanobot-ai

Successfully installed APScheduler-3.11.2 MarkupSafe-3.0.3 Pillow-12.2.0 XlsxWriter-3.2.9 aiohappyeyeballs-2.6.1 aiohttp-3.13.5 aiosignal-1.4.0 annotated-doc-0.0.4 annotated-types-0.7.0 anthropic-0.100.0 anyio-4.13.0 attrs-26.1.0 bidict-0.23.1 boto3-1.43.6 botocore-1.43.6 certifi-2026.4.22 cffi-2.0.0 chardet-5.2.0 charset_normalizer-3.4.7 click-8.3.3 colorama-0.4.6 croniter-6.2.2 cryptography-48.0.0 cssselect-1.4.0 ddgs-9.14.2 dingtalk-stream-0.24.3 distro-1.9.0 docstring-parser-0.18.0 dulwich-0.25.2 et-xmlfile-2.0.0 filelock-3.29.0 frozenlist-1.8.0 h11-0.16.0 httpcore-1.0.9 httpx-0.28.1 httpx-sse-0.4.3 idna-3.13 jinja2-3.1.6 jiter-0.14.0 jmespath-1.1.0 json-repair-0.59.6 jsonschema-4.26.0 jsonschema-specifications-2025.9.1 lark-oapi-1.5.5 loguru-0.7.3 lxml-6.1.0 lxml_html_clean-0.4.4 markdown-it-py-4.2.0 mcp-1.27.1 mdurl-0.1.2 msgpack-1.1.2 multidict-6.7.1 nanobot-ai-0.1.5.post3 oauth-cli-kit-0.1.3 openai-2.36.0 openpyxl-3.1.5 platformdirs-4.9.6 primp-1.2.3 prompt-toolkit-3.0.52 propcache-0.5.2 pycparser-3.0 pycryptodome-3.23.0 pydantic-2.13.4 pydantic-core-2.46.4 pydantic-settings-2.14.1 pygments-2.20.0 pyjwt-2.12.1 pypdf-5.9.0 python-dateutil-2.9.0.post0 python-docx-1.2.0 python-dotenv-1.2.2 python-engineio-4.13.1 python-multipart-0.0.27 python-pptx-1.0.2 python-socketio-5.16.1 python-telegram-bot-22.7 pywin32-311 pyyaml-6.0.3 qq-botpy-1.2.1 questionnaire-2.1.1 readability-lxml-0.8.4.1 referencing-0.37.0 regex-2026.5.9 requests-2.33.1 requests_toolbelt-1.0.0 rich-14.3.4 rpds-py-0.30.0 s3transfer-0.17.0 shellingham-1.5.4 simple-websocket-1.1.0 six-1.17.0 slack-sdk-3.41.0 slackify-markdown-0.2.2 sniffio-1.3.1 socksio-1.0.0 sse-starlette-3.4.2 starlette-1.0.0 tiktoken-0.12.0 tqdm-4.67.3 typer-0.25.1 typing-extensions-4.15.0 typing-inspection-0.4.2 tzdata-2026.2 tzlocal-5.3.1 urllib3-2.7.0 uvicorn-0.46.0 wcwidth-0.7.0 websocket-client-1.9.0 websockets-16.0 win32-setctime-1.2.0 wsproto-1.3.2 yarl-1.23.0

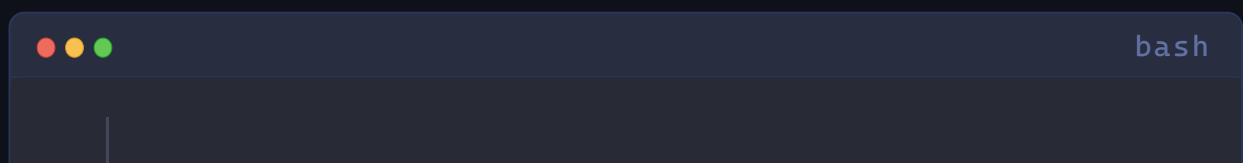
[notice] A new release of pip is available: 25.1.1 → 26.1.1

[notice] To update, run: python.exe -m pip install --upgrade pip

86180 ... \exec\exec1\nanobot-main main v3.13.5 14:41

2. 初始化 nanobot

在命令行窗口输入:



1 nanobot onboard

```
86180 ...\exec\exec1\nanobot-main  main v3.13.5 14:42
-> nanobot onboard
✓ Created config at C:\Users\86180\.nanobot\config.json
2026-05-10 14:42:58.860 | DEBUG | nanobot.channels.registry:discover_all:64 - Skipping
built-in channel 'matrix': Matrix dependencies not installed. Run: pip install nanobot-a
i[matrix]
Created AGENTS.md
Created HEARTBEAT.md
Created SOUL.md
Created TOOLS.md
Created USER.md
Created memory\MEMORY.md
Created memory\history.jsonl
2026-05-10 14:42:59.772 | INFO | nanobot.utils.gitstore:init:113 - Git store initiali
zed at C:\Users\86180\.nanobot\workspace

👤 nanobot is ready!

Next steps:
1. Add your API key to C:\Users\86180\.nanobot\config.json
   Get one at: https://openrouter.ai/keys
2. Chat: nanobot agent -m "Hello!" --config C:\Users\86180\.nanobot\config.json

Want Telegram/WhatsApp? See: https://github.com/HKUDES/nanobot#-chat-apps
```

3. 配置 nanobot

根据实验手册要求，在用户目录下的 `~/.nanobot/config.json` 配置 provider 与模型信息（注意：`apiKey` 属于敏感信息，报告与提交物中不应明文展示）。配置完成后保存即可。

4. 验证 nanobot 安装

```
bash
1 | nanobot agent
```

```
1: C:\Program Files\PowerShell\7\pwsh.exe X +
PowerShell 7.6.1
Loading personal and system profiles took 721ms.
86180 ~ v3.13.5 15:07
→
cd d:\MyCodeResource\bianyiyuanli\exec\exec1\nanobot-main
86180 ...\exec\exec1\nanobot-main ? main v3.13.5 15:08
→ .\env\Scripts\Activate.ps1
86180 ...\exec\exec1\nanobot-main ? main v3.13.5 15:09
→ nanobot agent
👤 Interactive mode (mimo-v2.5-pro) – type exit or Ctrl+C to quit

You: hi

👤 nanobot
你好！有什么需要帮忙的吗？

You: |
```

若能正常返回回复内容，说明 nanobot 安装、配置完成。



阶段三：自动化检测与缺陷修复

1. 准备工作

1.1 确认 clang.exe 路径

```
1 | "D:\MyCodeResource\bianyiyuanli\exec\exec1\llvm-project-llvmorg-17.0.5\llvm\cmake-build-release\bin\clang.exe"
```

1.2 确认规则信息

- 规则集名称： MISRA
- 规则ID： R_7_1、R_8_11、R_15_4

2. 使用自动化工具

2.1 进入自动化工具目录

```
1 | cd d:\MyCodeResource\bianyiyuanli\exec\exec1\自动化工具
```

2.2 运行自动化检测脚本

使用以下命令格式运行脚本：

```
bash
```

```
1 python nanobot_auto_checker.py --clang-path "<clang.exe 绝对路径>" --rule-set <规则集名称> --rule-id <规则 ID>
```

```
PowerShell 7.6.1
Loading personal and system profiles took 615ms.

86180 ...\bianyiyuanli\exec\exec1 main 15:41
cd d:\MyCodeResource\bianyiyuanli\exec\exec1\nanobot-main

86180 ...\exec\exec1\nanobot-main main v3.14.4 15:42
.\venv\Scripts\Activate.ps1

86180 ...\exec\exec1\nanobot-main main v3.13.5 15:42
cd d:\MyCodeResource\bianyiyuanli\exec\exec1\自动化工具

86180 ...\exec\exec1\自动化工具 main v3.13.5 15:42
python nanobot_auto_checker.py --clang-path "D:\MyCodeResource\bianyiyuanli\exec\exec1\llvm-project-llvmorg-17.0.5\llvm\cmake-build-release\bin\clang.exe" --rule-set MISRA --rule-id R_7_1
规则集: MISRA
规则 ID: R_7_1
违规用例 (violate): R_7_1_violate.cpp -> 报出 MISRA.R_7_1 缺陷
合规用例 (comply): R_7_1_comply.cpp -> 不应报出 MISRA.R_7_1 缺陷
漏报边界 (fn): R_7_1_fn.cpp -> 报出 MISRA.R_7_1 缺陷
误报边界 (fp): R_7_1_fp.cpp -> 不应报出 MISRA.R_7_1 缺陷
-----

--- 违规用例 (violate) ---

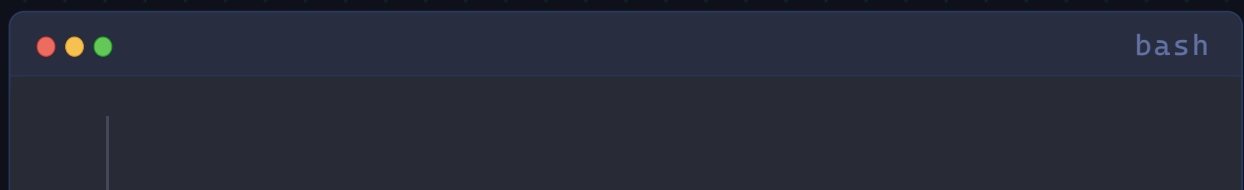
🤖 nanobot
以下是完整的执行输出:

| STDOUT: (无)

| STDERR:

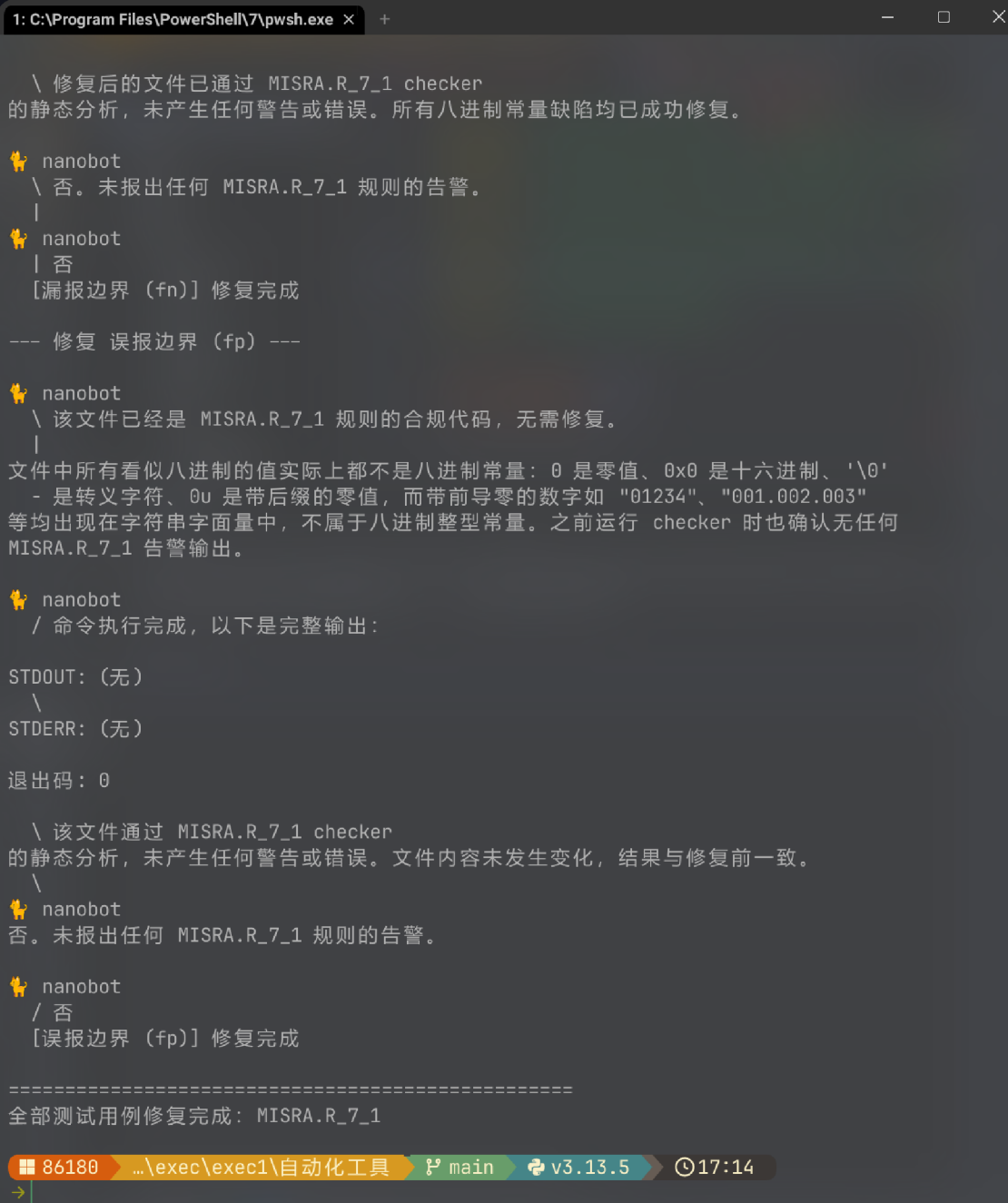
-
\ D:\MyCodeResource\bianyiyuanli\exec\exec1\自动化工具\用例库副本\MISRA\R_7_1_violate.cpp:3:14: warning: Octal constants shall not be used (Rule 7.1)
- [MISRA.R_7_1]
/ 3 | int a = 010; /* 八进制 010 = 十进制 8 */
| ^
D:\MyCodeResource\bianyiyuanli\exec\exec1\自动化工具\用例库副本\MISRA\R_7_1_violate.cpp:4:14: warning: Octal constants shall not be used (Rule 7.1)
[MISRA.R_7_1]
4 | int b = 077; /* 八进制 077 = 十进制 63 */
| ^
D:\MyCodeResource\bianyiyuanli\exec\exec1\自动化工具\用例库副本\MISRA\R_7_1_violate.cpp:5:14: warning: Octal constants shall not be used (Rule 7.1)
[MISRA.R_7_1]
5 | int c = 0123; /* 八进制 0123 = 十进制 83 */
| ^
\
D:\MyCodeResource\bianyiyuanli\exec\exec1\自动化工具\用例库副本\MISRA\R_7_1_violate.cpp:6:14: warning: Octal constants shall not be used (Rule 7.1)
```

本实验针对 3 条规则分别执行自动化脚本:



```
1 python nanobot_auto_checker.py --clang-path "<clang  
   路径>" --rule-set MISRA --rule-id R_7_1  
2 python nanobot_auto_checker.py --clang-path "<clang  
   路径>" --rule-set MISRA --rule-id R_8_11  
3 python nanobot_auto_checker.py --clang-path "<clang  
   路径>" --rule-set MISRA --rule-id R_15_4
```

3. 自动化工具工作流程



```
1: C:\Program Files\PowerShell\7\powershell.exe x +  
  
  \ 修复后的文件已通过 MISRA.R_7_1 checker  
  的静态分析，未产生任何警告或错误。所有八进制常量缺陷均已成功修复。  
  
  🤖 nanobot  
  \ 否。未报出任何 MISRA.R_7_1 规则的告警。  
  |  
  🤖 nanobot  
  | 否  
  | [漏报边界 (fn)] 修复完成  
  
  --- 修复 误报边界 (fp) ---  
  
  🤖 nanobot  
  \ 该文件已经是 MISRA.R_7_1 规则的合规代码，无需修复。  
  |  
  文件中所有看似八进制的值实际上都不是八进制常量：0 是零值、0x0 是十六进制、'\0'  
  - 是转义字符、0u 是带后缀的零值，而带前导零的数字如 "01234"、"001.002.003"  
  等均出现在字符串字面量中，不属于八进制整型常量。之前运行 checker 时也确认无任何  
  MISRA.R_7_1 告警输出。  
  
  🤖 nanobot  
  / 命令执行完成，以下是完整输出：  
  
  STDOUT: (无)  
  \  
  STDERR: (无)  
  
  退出码：0  
  
  \ 该文件通过 MISRA.R_7_1 checker  
  的静态分析，未产生任何警告或错误。文件内容未发生变化，结果与修复前一致。  
  \  
  🤖 nanobot  
  否。未报出任何 MISRA.R_7_1 规则的告警。  
  
  🤖 nanobot  
  / 否  
  | [误报边界 (fp)] 修复完成  
  
  =====  
  全部测试用例修复完成：MISRA.R_7_1  
  
  86180 ... \exec\exec1\自动化工具 main v3.13.5 17:14  
  →
```

-
/ 验证结果：重新运行 MISRA.R_8_11 检查，无任何警告输出，退出码为 0，缺陷已消除。

👤 nanobot

- 命令执行完成，退出码为 0。

-

输出结果：无任何输出（无警告、无错误）。

|

修复后的 R_8_11_fn.cpp 文件已通过 MISRA.R_8_11 检查，不再报出任何违规信息。

- 👤 nanobot

否。MISRA.R_8_11 规则的告警未被报出，输出中无任何警告信息。

👤 nanobot

否

[漏报边界 (fn)] 修复完成

--- 修复 误报边界 (fp) ---

\

👤 nanobot

- 该文件不存在 MISRA.R_8_11 规则的违规。文件中的数组 local_buf 使用了 static
- 修饰符，具有内部链接 (internal linkage)，而 MISRA Rule 8.11
/ 仅适用于外部链接 (external
\ linkage) 的数组声明。之前的检查结果也证实该文件未报出任何 MISRA.R_8_11
/ 告警，无需修复。

\

- 👤 nanobot

命令执行完成，退出码为 0。

|

- 输出结果：无任何输出（无警告、无错误）。

\

| 该文件通过了 MISRA.R_8_11 检查，未报出任何违规信息。

/

👤 nanobot

否。MISRA.R_8_11 规则的告警未被报出，输出中无任何警告信息。

👤 nanobot

/ 否

[误报边界 (fp)] 修复完成

=====
全部测试用例修复完成：MISRA.R_8_11

```
1: C:\Program Files\PowerShell\7\pwsh.exe x +
- 否
[漏报边界 (fn)] 修复完成

--- 修复 误报边界 (fp) ---

| 🤖 nanobot
/ 该文件 R_15_4_fp.cpp 中不存在 MISRA.R_15_4 规则的缺陷，无需修复。

文件中每个循环最多只使用了一个 break，且没有 goto:

\ • test_fp_nested_loops: 内层循环（第 10 行）和外层循环（第 14 行）各含一个
break，每个循环独立计算，均不超过一个。
• test_fp_separate_same_scope: 两个独立循环各含一个 break，互不影响。

- 该文件本身就是无误报（false positive）的合规用例，代码无需改动。

/ 🤖 nanobot
以下是命令的完整输出：

标准输出 (stdout): (空)

/ 标准错误 (stderr): (空)

退出码: 0

| -----

/ 分析结果摘要:

- • 未检出任何 MISRA.R_15_4 告警或错误。
• 程序正常退出（退出码 0），输出全部为空。
• 该文件本身即为合规用例，不存在 MISRA.R_15_4 缺陷。

| 🤖 nanobot
否，未报出任何 MISRA.R_15_4 告警。

/ 🤖 nanobot
\ 否
[误报边界 (fp)] 修复完成

=====
全部测试用例修复完成: MISRA.R_15_4

86180 ...\exec\exec1\自动化工具 8? main v3.13.5 17:29
→
```

4. 自动化检测与修复结果（产物说明）

自动化工具的核心产物包括：

- 用例库：为每条规则生成四类用例（violate/comply/fn/fp）
- 用例库副本：在副本中进行缺陷修复（原始用例库不会被修改）
- 关联关系文件：记录“规则 ↔ 用例文件 ↔ 预期结果”的映射

本实验已将阶段二/三产物按提交要求归档到：

- 实验报告/阶段二产物/用例库/MISRA/

- 实验报告/阶段二产物/用例库副本/MISRA/
- 实验报告/阶段二产物/关联关系.txt

并额外归档了自动化脚本与规则集描述文件，便于复现：

- 实验报告/阶段二产物/自动化工具/

产物归档与提交结构

```
1  实验报告/
2  └─ 实验报告.md
3  └─ 实验报告.assets/           # 报告截
   图
4  └─ 阶段一产物/
5  │   └─ 源文件/               # 阶段一
   源文件
6  │   │   └─ MISRAC_7_1_Checker.cpp
7  │   │   └─ MISRAC_8_11_Checker.cpp
8  │   │   └─ MISRAC_15_4_Checker.cpp
9  │   └─ 构建产物/
10  │       └─ clang.exe
11  └─ 阶段二产物/
12  │   └─ 用例库/               # 阶段二
   产物
13  │       └─ MISRA/
14  │   └─ 用例库副本/           # 阶段
   二产物
15  │       └─ MISRA/
16  │   └─ 关联关系.txt         # 阶段
   二产物
17  └─ 自动化工具/             # 复现
   实验
18  └─ nanobot_auto_checker.py
19  └─ README.md
```

遇到的问题与解决

1) 自动生成用例需要兼容 C++ 解析

自动化工具生成的用例文件扩展名为 `.cpp`，默认按 C++ 语法规则解析，因此生成代码时必须避免会直接导致编译失败的写法（例如不完整数组定义 `static int a[];`）。解决方式是在生成提示中明确约束：

- 必须生成可通过 `clang --analyze` 的代码（无 error）
- 若省略数组大小，必须提供初始化器（如 `static int a[] = {0};`）或使用 `extern` 声明

2) 提示词模板与花括号转义

自动化脚本的提示词模板使用字符串格式化机制；当提示词中需要出现字面量花括号（例如展示 C/C++ 初始化器 `{0}`），需要写为 `{{0}}` 才能避免格式化时报错。